

0714 arteamis meeting

Questions:

1. I don't know whether we need a Desktop app for the personal brain. It should be a webpage also right? Bcz if we make it an app, user have to install it first. This is not very friendly, I hope I can just sign up and log in to get connected to my personal brain.

Rn, for the wedge, it's a web app: sign up, log in, done.

yeah actually personal brain can lives in the cloud, private to you (using tenant-isolated, invisible to your team until you are promoted) and the desktop app is a later, optional thing, but only for one reason: some security-sensitive studios will not put raw secrets or client code in any cloud, ever so that's what desktop's Local Vault is for.

2. Should we provide services for personal users? They don't have a studio or group. Just want to find a place to store their skills and standards, so they don't have to tell agents over and over again. I think we should, but this is not our core selling point. We can do sth like always notify users that he/she could join a group.

can't be better. Yes, I agree with both sentences

3. What is a lesson exactly for our system? A .md file? I know we need to add source along with trade-offs and everything to make it shareable, but what is a lesson to the machine? I want to know this bcz If I just want to use some lesson for myself, I don't want to share with any other people, what should I add to the source? Can I skip writing trade-offs and everything?

In the story below, Alice fills in tradeoff / owner / next-step when she saves the lesson. In the real system, those fields live on the Rule, not the Lesson:

- A Lesson (shared_lesson_proposal) = the insight text in your words + the source citation + a trust level.
=> This has no tradeoffs.
- A Rule (team_rule) = the standard built from accepted lessons, and this is what carries tradeoff_accepted, next_execution_step, review_at, and evidence links.

So:

Lesson = a source-backed claim.

Rule = a standard with the operational metadata.

everything else in the story (the source-review gate, "human-asserted" trust level, secrets auto-denied, propose-only agent, trace with cost, learning-can't-auto-save) is exactly right.

4. After clicking ask, how can I convert a source to lesson, what should I talk about with the chatbox? Is it a complete example?

User Story: Alice is an engineer, Bob is a Checker, and the team is developing Web3 smart contracts for Stellar.

On the first morning, Alice receives a security audit report.

She uploads **stellar-audit-2025.pdf** to Artemis (via **Add Source**). After the system finishes indexing it asynchronously, she receives a notification.

Alice asks a question:

On the **Ask** page, she selects the uploaded source and enters the following prompt in the chatbox:

"Does the `withdraw` function have any security issues?"

The system responds:

"A reentrancy vulnerability (HIGH) is identified in § 3.2 on page 14. The `withdraw()` function performs an external call before updating the user's balance."

 **Citation:** *stellar-audit-2025.pdf* · p.14 § 3.2

Alice decides this knowledge should be preserved as a Lesson.

She clicks **Save as Lesson** and writes, in her own words:

"The Stellar contract's `withdraw()` function is vulnerable to reentrancy. The audit rates this issue as HIGH and it should be fixed."

She fills in the metadata:

- **Tradeoff:** Fixing the issue requires refactoring `withdraw()`, which may increase gas costs.
- **Owner:** Alice
- **Next step:** Add `ReentrancyGuard` in the next sprint.

She then selects **Share to Group for Review**.

Bob (the Checker) receives a notification and opens the **Review Queue**.

He sees:

- **Lesson:** `withdraw()` has a HIGH-severity reentrancy vulnerability.
- **Source:** *stellar-audit-2025.pdf* · p.14 § 3.2
- **Trust level: Human-asserted** (written by Alice, not generated solely by AI)

Bob opens the original document (**Open Source Document**) and confirms that page 14, § 3.2 indeed describes the issue. He checks **"I have read the source"** and clicks **Accept**.

✅ The Lesson is now promoted to a **Group Lesson** and becomes part of the **Company Brain**.

The team leader decides to promote this Lesson into a Rule.

This is no longer just something the team knows—it becomes a standard that every future contract must follow.

Rule:

All functions involving ETH or token transfers must use OpenZeppelin's `ReentrancyGuard`.
Any pull request that omits it without a documented exception must not be merged.

The Rule includes:

- **Evidence:** the approved Lesson above
 - **Owner:** Alice
 - **Review date:** three months later
 - **Linked work item:** the remediation task scheduled for the next sprint
-

Engineer Carol wants an Agent to review a new smart contract.

She creates a **Context Pack** named **"Token Contract Review"**, containing:

✅ **Allowed**

- Sections § 3– § 5 of *stellar-audit-2025.pdf* (the relevant chapters)
- The Rule: *"All transfer functions must use `ReentrancyGuard`."*

❌ Denied

- `wallet-seed-backup.txt` (contains private keys; automatically rejected because the system detects secrets)
- `personal-notes-carol.txt` (Carol's personal notes; not promoted, therefore not visible to Agents)

This is **Denied Context**:

It is not that the Agent *doesn't want* to read these files—the governance policy explicitly prevents access, and the reasons are clearly documented in the Context Pack.

Carol creates a **Handoff** (a task contract for the Agent).

A Handoff serves as the Agent's **task specification and permission boundary**.

It contains:

- **Objective:** "Check whether `withdraw()` in `contracts/token.sol` complies with the Rule."
- **Context Pack:** *Token Contract Review* (3 allowed sources, 2 denied items with documented reasons)
- **Write mode: Propose-only** (the Agent may suggest changes but cannot modify the code directly)
- **Budget:** Maximum \$5
- **Stop condition:** "List any issues found. Do not fix them automatically."

Carol clicks **Approve & Launch**.

After the Agent finishes, Carol opens the **Trace**.

The Trace records:

- Which files the Agent accessed (including the specific pages of the Stellar audit and the Rule text)
- Which files the Agent was denied access to (`wallet-seed-backup.txt` , reason: **secrets**; `personal-notes-carol.txt` , reason: **private**)
- The Agent's proposed change: add the `nonReentrant` modifier to `withdraw()`
- Total cost: **\$1.24**

Carol reviews the diff in the **Proposed Writes** tab and clicks **Approve Write**.

The Agent also suggests a new Learning:

"Here's an example of how `ReentrancyGuard` can be used within an inheritance hierarchy... (code snippet)"

This suggestion cannot be added to the Company Brain automatically.

Carol clicks **Review as Proposal**, sending it to the **Review Queue**, where Bob reviews it through the same approval workflow.

End-to-End Knowledge Flow

代码块

```
1 stellar-audit-2025.pdf (Source)
2     ↓
3     Ask (Q&A)
4     ↓
5 Alice extracts a Lesson (Human-asserted)
6     ↓
7 Bob reviews it (Group Lesson, Source-backed)
8     ↓
9 Leader promotes it to a Rule (organization-wide policy)
10    ↓
11 Carol creates a Context Pack (Allowed + Denied context)
12    ↓
13 Carol creates a Handoff (preflight task contract)
14    ↓
15 Agent executes (within explicit boundaries, with full logging)
16    ↓
17 Trace (records every action)
18    ↓
19 Human reviews Proposed Writes
20    ↓
21 Agent submits a Learning proposal
22    ↓
23 Back to the Review Queue
```

5. Should we allow personal user to convert their personal lessons into personal rules? I am not sure whether we have this already.

i haven't built that feature, but technically, we can. I think we need to resolve this problem in the meeting.

6. I think we should separate the functions of teams into "groups". e.g. I am an CEO of a studio, we have R&D, we have marketing, we have product. They all have different global rules. I should switch the rules by switching tenants right?

Yes, we should split teams into groups. To be true: you switch by group, not by tenant.

Let me explain:

- Tenant = your whole studio. It's the security wall - it's what makes sure nobody outside your studio can ever see your data. You would only ever "switch tenant" if you actually belonged to two different companies.
- Group (a "workspace" internally) = one team inside your studio - R&D, Marketing, Product.

So R&D / Marketing / Product are three groups living inside your one studio (one tenant). When you want to see R&D's rules vs Marketing's rules, you switch group - not tenant.

And rules can sit at two levels:

- Company-wide rules → apply to every group and every agent (e.g. "never commit secrets").
- Group rules → only that team (e.g. R&D's code conventions).

So someone in R&D sees the company-wide rules plus R&D's own rules.

7. Is it a mandatory process to ask the source? What if user has already extract the main learning themselves? Is it allowed?

Not mandatory because ask is a helper, not a gate. If you already know the lesson, just write it directly, you don't have to ask the chatbox first.

The difference is the trust level the reviewer sees:

- Lesson made from an Ask answer (with citation) → "source-backed" - strongest.
- Lesson you type from your own head (no source) → "human-asserted" - a person vouches for it, but there's no document behind it.

Both are allowed (the lesson's source citation is optional in the model). The reviewer just sees which kind it is and decides accordingly. For a high-stakes claim, they might ask you to attach a source.

So self-extracted learnings are fine; they're just labeled as assertions rather than source-backed.

8. Is it mandatory to start creating every rule from add source as a leader? Bcz if he just want to add a global rule like "commit message has to be start with Jira-xxx" there is no need to provide source and lesson and trade-off?

Agree, so i think we could go this way:

add a "lightweight rule" path, so a leader states the rule. It's active for the group and its agents, no source or tradeoff required.

keep the heavy evidence + tradeoff + review flow for consequential rules (like the security one in Alice's story).

Two tiers: convention rules (just the statement) and evidence-backed rules (the governed kind).

9. So for each devs or makers, they can either give tasks on our platform as handoffs. Or they can user their own IDEs and still be governed by the context packs right?

Yes, both and that's the whole point.

Two paths:

1. On-platform Handoff (full governance): preflight contract, propose-only write mode, budget, stop condition, trace, and a proposed-writes review.

-> Arteamis runs the whole thing.

2. Your own IDE (Cursor / Claude Code) via MCP (Arteamis serves the governed context pack to IDE):

the allowed context, the denied context (secrets auto-blocked, un-promoted notes hidden), and the rules/skills. Your agent works inside that scoped boundary without leaving your editor.

